



Name : Sharon Elango

Batch : 3 Roll No : 62

Expt.-9: Apply PCA and LDA to reduce dataset dimensions

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import time

from sklearn import datasets
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis

from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
```

```
In [2]: #Load dataset
iris = datasets.load_iris()
X = iris.data
y = iris.target

feature_names = iris.feature_names
print(iris.feature_names)
print()
print(iris.target_names)

['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']

['setosa' 'versicolor' 'virginica']
```

Part - 1 : Dataset Summary Table

```
In [3]: print("DATASET SUMMARY")
print("Number of samples:", X.shape[0])
print("Number of features:", X.shape[1])
print("Number of classes:", len(np.unique(y)))
print(feature_names)
print()
```

DATASET SUMMARY

Number of samples: 150

Number of features: 4

Number of classes: 3

['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']

Part - 2 : Feature Statistics Table (before PCA):

```
In [4]: #Convert to dataframe for statistics
df = pd.DataFrame(X, columns=feature_names)

#Table 1: Feature Statistics
print("FEATURE STATISTICS")
stats_table = pd.DataFrame({
    "Mean": df.mean(),
    "Standard Deviation": df.std()
})
print(stats_table)
print()
print(df.head())
print()
```

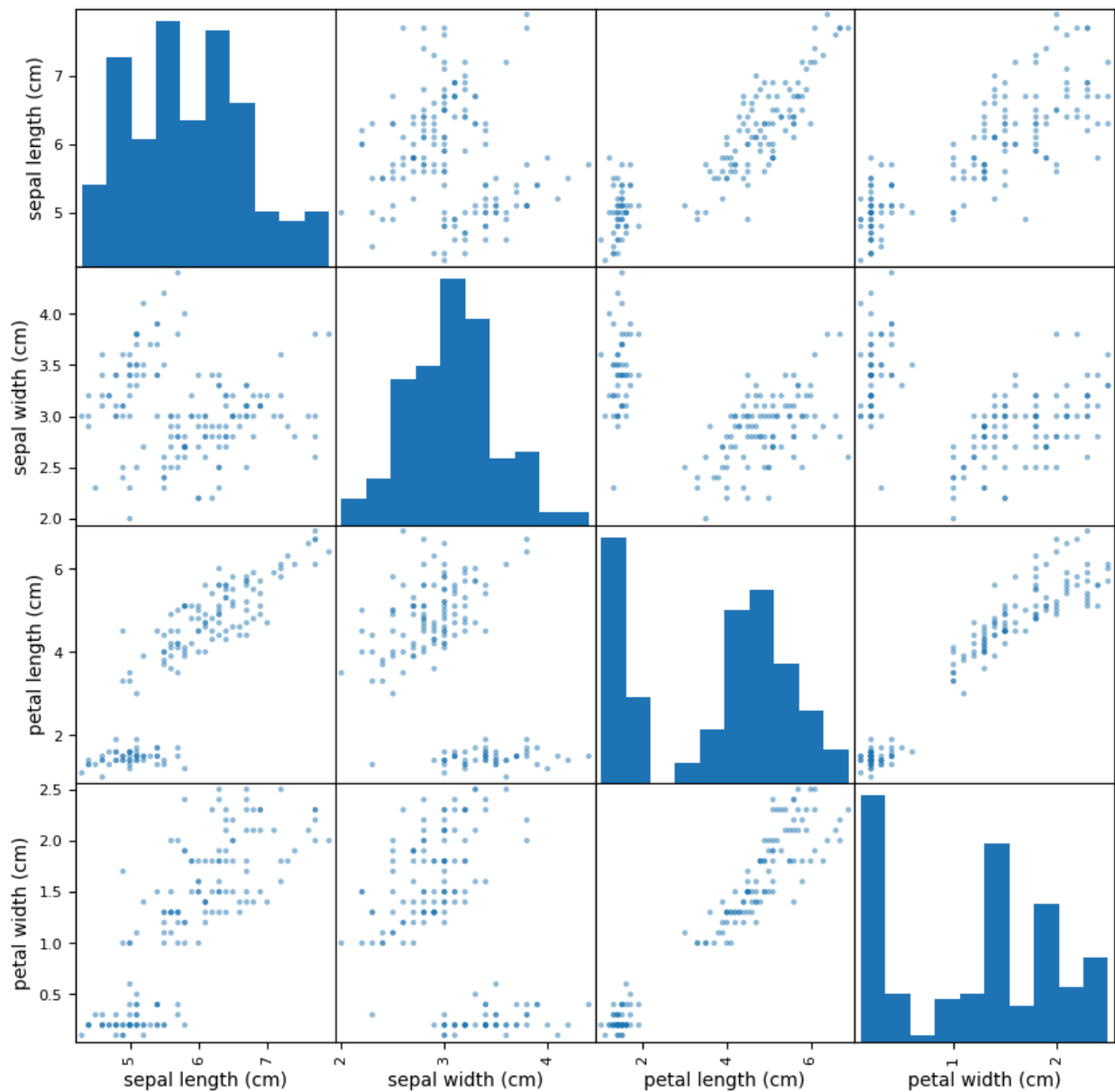
FEATURE STATISTICS

	Mean	Standard Deviation
sepal length (cm)	5.843333	0.828066
sepal width (cm)	3.057333	0.435866
petal length (cm)	3.758000	1.765298
petal width (cm)	1.199333	0.762238

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

```
In [7]: #pairwise scatter plots
pd.plotting.scatter_matrix(
    df.iloc[:,0:4],
    figsize=(10,10),
    diagonal='hist'
)

plt.show()
```



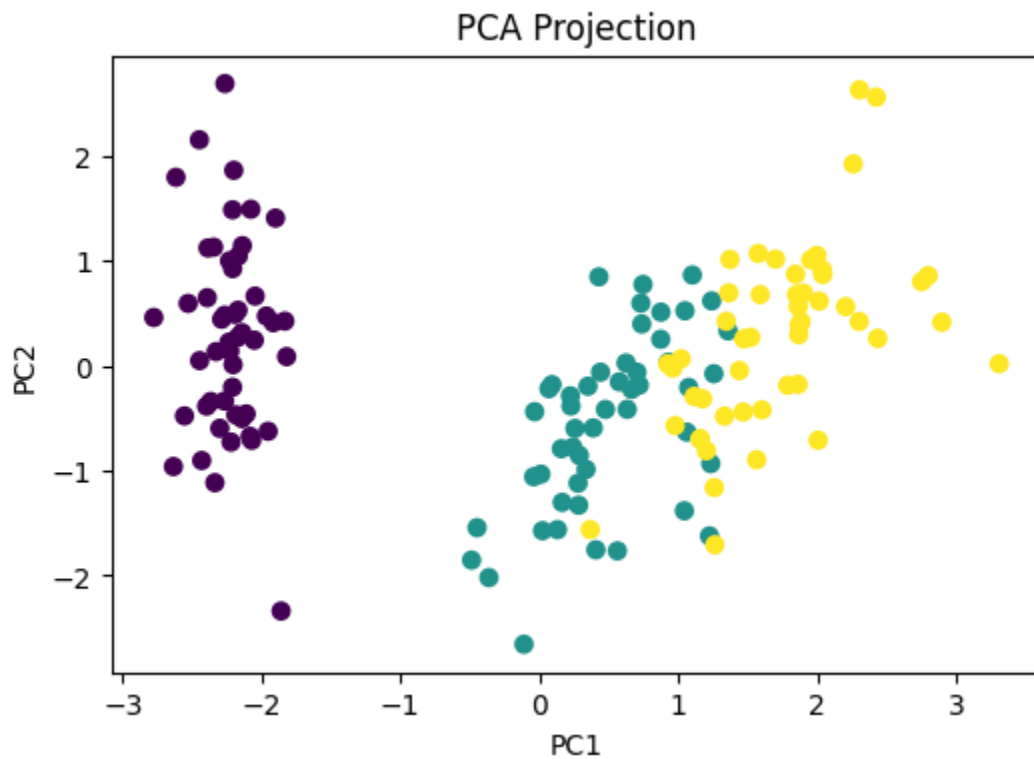
Part - 3 : PCA

```
In [9]: #Standardize
sc = StandardScaler()
X_scaled = sc.fit_transform(X)
```

```
In [10]: pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
```

```
In [12]: #Plot PCA
plt.figure(figsize=(6,4))
plt.scatter(X_pca[:,0], X_pca[:,1], c=y)
plt.title("PCA Projection")
plt.xlabel("PC1")
```

```
plt.ylabel("PC2")
plt.show()
```



Calculate Variance

```
In [16]: explained_variance = pca.explained_variance_
variance_ratio = pca.explained_variance_ratio_
cumulative_variance = np.cumsum(variance_ratio)
```

```
In [18]: pca_table = pd.DataFrame({
    "Eigenvalue": explained_variance,
    "Explained variance (%)": variance_ratio*100,
    "Cumulative variance (%)": cumulative_variance*100
})

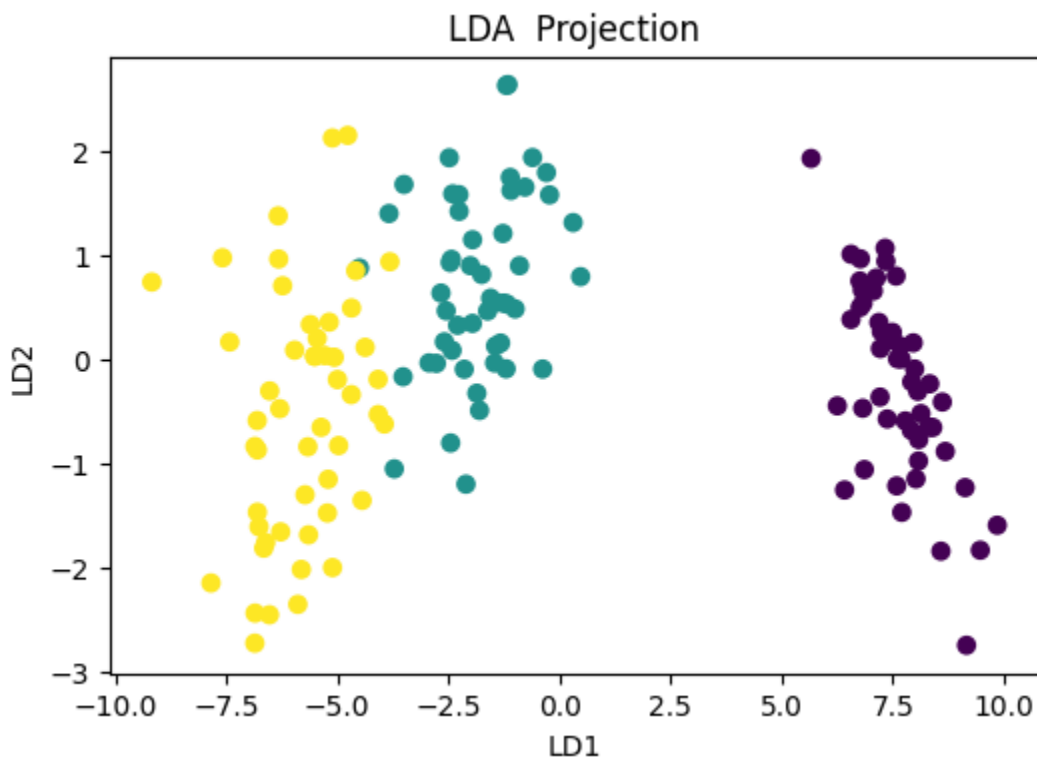
print("PCA VARIANCE TABLE")
print(pca_table)
print()
```

```
PCA VARIANCE TABLE
   Eigenvalue  Explained variance (%)  Cumulative variance (%)
0    2.938085             72.962445             72.962445
1    0.920165             22.850762             95.813207
```

Part - 4 : LDA

```
In [19]: # LDA
lda = LinearDiscriminantAnalysis(n_components=2)
X_lda = lda.fit_transform(X_scaled, y)
```

```
In [20]: #Plot LDA
plt.figure(figsize=(6,4))
plt.scatter(X_lda[:,0], X_lda[:,1], c=y)
plt.title("LDA Projection")
plt.xlabel("LD1")
plt.ylabel("LD2")
plt.show()
```



Part - 5 : checking KNN Accuracy using PCA and LDA data

```
In [21]: #Train-test split
X_train, X_test, y_train, y_test = train_test_split (X_scaled, y, test_size=0.3)
```

```
In [22]: Xp_train, Xp_test, yp_train, yp_test = train_test_split (X_pca, y, test_size=0.3)
```

```
In [23]: Xl_train, Xl_test, yl_train, yl_test = train_test_split (X_lda, y, test_size=0.3)
```

```
In [31]: #Classifier
model = KNeighborsClassifier()

#Accuracy Original
start = time.time()
model.fit(X_train,y_train)
train_time_original = time.time() - start

y_pred = model.predict(X_test)
acc_original = accuracy_score(y_test, y_pred)

#Accuracy PCA
start = time.time()
model.fit(Xp_train, yp_train)
train_time_pca = time.time() - start

y_pred = model.predict(Xp_test)
acc_pca = accuracy_score(y_test, y_pred)
```

```
In [34]: #Accuracy LDA
start = time.time()
model.fit(Xl_train, yl_train)
train_time_lda = time.time() - start

y_pred = model.predict(Xl_test)
acc_lda = accuracy_score(y_test, y_pred)
```

```
In [37]: # Accuracy table
accuracy_table = pd.DataFrame ({
    "Dataset Version": ["Original", "PCA Reduced", "LDA Reduced"],
    "Accuracy": [acc_original, acc_pca, acc_lda],
    "Training Time (seconds)": [train_time_original, train_time_pca, train_time_lda]
})

print("MODEL PERFORMANCE TABLE")
print(accuracy_table)
```

```
MODEL PERFORMANCE TABLE
  Dataset Version  Accuracy  Training Time (seconds)
0      Original    1.000000             0.000614
1   PCA Reduced    0.955556             0.000445
2    LDA Reduced    1.000000             0.000823
```